

סרטייה

מערכת השאלת סרטי וידאו/DVD

הגשה סופית · מסמך 2 מתוך 3

מסמך תכנון

Ahmad Abd Al-Haleem	מגיש
סדנה ליישומים בתכנות Java	קורס
האוניברסיטה הפתוחה	מוסד
אוגוסט 2026	תאריך

github.com/Ahmed95zx/Sartia

תוכן המסמך

1. תיאור כללי	8. שכבת הנתונים
2. מינוחים	9. שכבת הלוגיקה העסקית
3. הטכנולוגיות שבשימוש	10. שכבת התצוגה
4. ארכיטקטורת המערכת	11. שכבת ה-REST
5. בסיס הנתונים	12. טיפול במקביליות
6. מבנה החבילות	13. אבטחה
7. שכבת המודל	14. בדיקות
	15. החלטות תכנון והנימוק להן

1. תיאור כללי

מסמך זה מתאר את המבנה הפנימי של מערכת "סרטייה", את הטכנולוגיות שבהן מומשה, ואת המחלקות והקשרים ביניהן. המערכת בנויה כיישום Web בשלוש שכבות (MVC) מעל Jakarta EE 10, עם בסיס נתונים MySQL. הדגש התכנוני המרכזי הוא הפרדה בין השכבות ונכונות בתנאי ריבוי משתמשים. ממשק המשתמש, נתוני הקטלוג והודעות השגיה כתובים באנגלית, והפריסה היא משמאל לימין. מסמך זה כתוב בעברית ומצטט מחרזות שמופיעות על המסך בלשונן.

2. מינוחים

מונח	משמעות
סרט (Movie)	כותר בקטלוג. אינו ניתן להשאלה בעצמו.
עותק (Copy)	דיסק פיזי אחד של סרט. זוהי היחידה המושאלת בפועל.
השאלה (Rental)	רשומה המתעדת שעותק יצא ללקוח. "פתוחה" כל עוד לא הוחזר.
לקוח	משתמש רשום המשאיל סרטים.
מנהלן	משתמש בעל הרשאות ניהול קטלוג ומלאי.

ההבחנה בין סרט לעותק היא ההחלטה המבנית החשובה ביותר במערכת. היא מאפשרת להחזיק כמה עותקים של כותר מבוקש, ובו בזמן מאפשרת לאכוף במדויק שעותק מסוים לא יושאל פעמיים.

3. הטכנולוגיות שבשימוש

הפרויקט מממש **שלוש** מהטכנולוגיות שנלמדו בקורס:

JSF — Jakarta Faces 4.0 3.1

מסגרת העבודה של ממשק המשתמש. דפי XHTML עם Expression Language הקשורים ל-Backing Beans בצד השרת. בשימוש: תבנית משותפת (`ui:insert` / `ui:composition`), רכיבי טפסים, `ui:repeat`, המרות תאריך ומספר, הצגת הודעות ואימות קלט. ניהול המצב נעשה באמצעות `@ViewScoped` ו-`@SessionScoped`.

JDBC 3.2

הגישה לבסיס הנתונים. כל השאילתות נכתבו ידנית כ- `PreparedStatement`, ללא מיפוי אוטומטי. הבחירה ב-JDBC ולא ב-JPA מכוונת: המנגנון המרכזי במערכת הוא נעילת שורות מפורשת (`SELECT ... FOR UPDATE SKIP LOCKED`), וכתיבתו ישירות ב-SQL ברורה ומדויקת יותר מאשר דרך שכבת הפשטה. מנוהל באמצעות מאגר חיבורים (HikariCP) וגבולות טרנזקציה מפורשים.

REST — Jakarta RESTful Web Services 3.1 (JAX-RS) 3.3

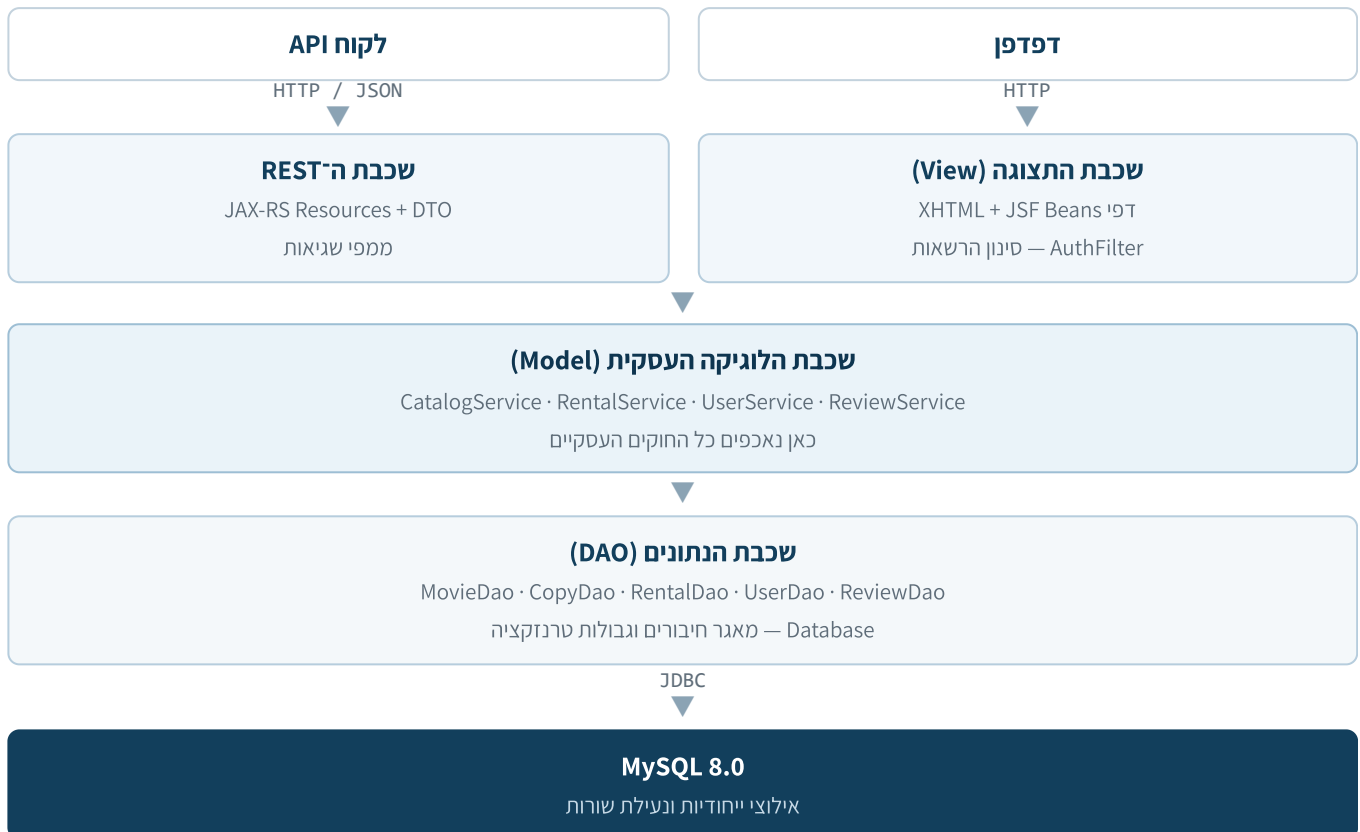
ממשק תכנות המחזיר JSON, בכתובת `/api`. חושף את הקטלוג לקריאה. משתמש באותה שכבת שירות של דפי ה-JSF, ולכן אינו יכול לסתור אותם. כולל ממפי חריגות (`ExceptionHandler`) הממירים חריגות עסקיות לקודי HTTP מתאימים.

3.4 טכנולוגיות תומכות

רכיב	גרסה	תפקיד
Java	17 (LTS)	שפת המימוש
Jakarta EE	10	Faces, CDI, JAX-RS, Servlet
CDI	4.0	הזרקת תלויות וניהול מחזור חיים
MySQL	8.0	בסיס הנתונים
HikariCP	5.1	מאגר חיבורים
Payara Micro	6.2024.6	שרת היישומים
Maven	3.9	בנייה וניהול תלויות
JUnit 5	5.10	בדיקות

4. ארכיטקטורת המערכת

4.1 תרשים כללי



4.2 כללי ההפרדה בין השכבות

ההפרדה אינה הצהרתית בלבד; היא נאכפת על ידי כיווני התלות:

- **שכבת התצוגה אינה ניגשת ל-DAO.** אין ב-Backing Bean אף ייבוא מחבילת `dao`. כל פעולה עוברת דרך שירות.
- **שכבת השירות אינה מכירה HTTP.** אין בה `HttpServletRequest`, אין `FacesContext` ואין קודי סטטוס. היא זורקת חריגות מתחום העסק, וכל שכבת כניסה מתרגמת אותן לדרכה — JSF להודעה על המסך, JAX-RS לקוד HTTP.
- **שכבת ה-DAO אינה מכירה חוקים עסקיים.** היא מבצעת שאילתות ומחזירה אובייקטי מודל. ההחלטה "האם מותר להשאיל" נמצאת בשירות בלבד.
- **שתי שכבות הכניסה שוות מעמד.** דפי ה-JSF ומשאבי ה-REST הם שני לקוחות של אותה שכבת שירות. מכאן שחוק כמו מכסת ההשאלות נאכף פעם אחת בלבד, ולא ניתן לעקוף אותו דרך ממשק אחר.

5. בסיס הנתונים

5.1 גישה

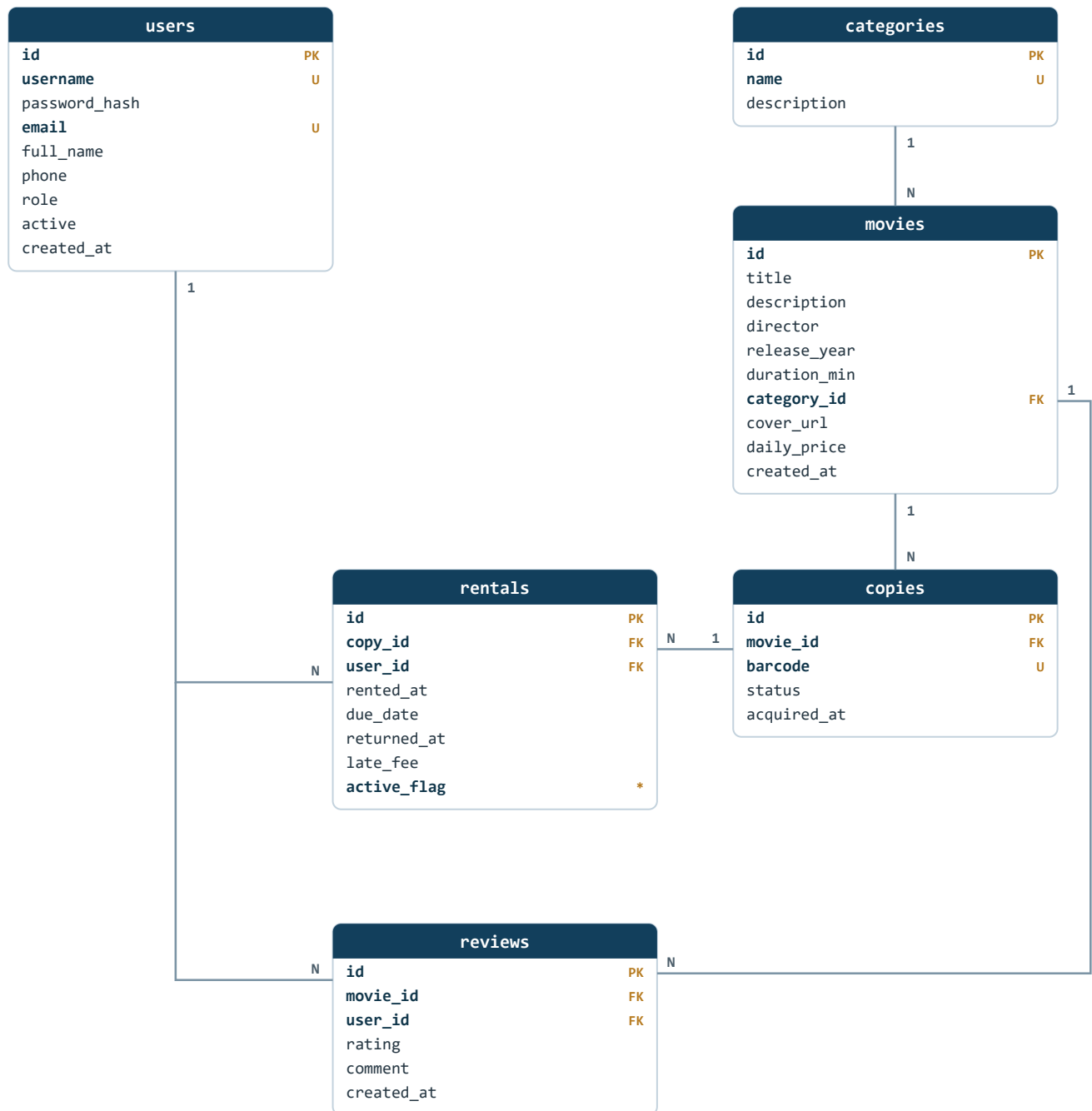
בסיס הנתונים הוא MySQL 8.0 המאזין על פורט 3306. הגישה אליו נעשית בתקן JDBC דרך מחלקת `Database`, המחזיקה מאגר חיבורים. פרטי החיבור נמצאים בקובץ `sartia.properties` וניתנים לדריסה בפרמטר הרצה.

מפתח	תיאור	טבלה
id	משתמשי המערכת – לקוחות ומנהלים, כולל תמצית הסיסמה והתפקיד.	users
id	קטגוריות הסרטים. נטענות עם הסכמה.	categories
id	כותרת הקטלוג. מצביעה על categories. כוללת את cover_url.	movies
id	העותקים הפיזיים. מצביעה על movies. היחידה המושאלת.	copies
id	רשומת השאלה – עותק, לקוח, תאריכים וקנס.	rentals
id	ביקורות לקוחות. ייחודי לצמד (סרט, לקוח).	reviews

תמונות העטיפה – נתיב ולא תוכן. העמודה `cover_url` שומרת **מצביע** לתמונה ולא את הבתים שלה. שמירת התמונות עצמן בבסיס הנתונים (`BLOB`) הייתה מנפחת כל שאילתת קטלוג בנתונים שאיש אינו זקוק להם בשלב הרשימה, ומונעת מהדפדפן ומהשרת להשתמש במטמון הרגיל של קבצים סטטיים. הקבצים עצמם יושבים תחת `src/main/webapp/images/covers/` ונארזים לתוך ה-WAR, ולכן הם **חלק מהיישום ואינם תלויים בשרת חיצוני** – הקטלוג נראה תקין גם ללא חיבור לאינטרנט, שיקול מעשי לקראת הדגמה חיה.

הערך יכול להיות נתיב יחסי ליישום (`/images/covers/1.jpg`) או כתובת מלאה (`https://...`). שכבת התצוגה מבחינה בין השניים ומרכיבה את הכתובת הסופית, כך ששתי השיטות יכולות להתקיים זו לצד זו באותו קטלוג.

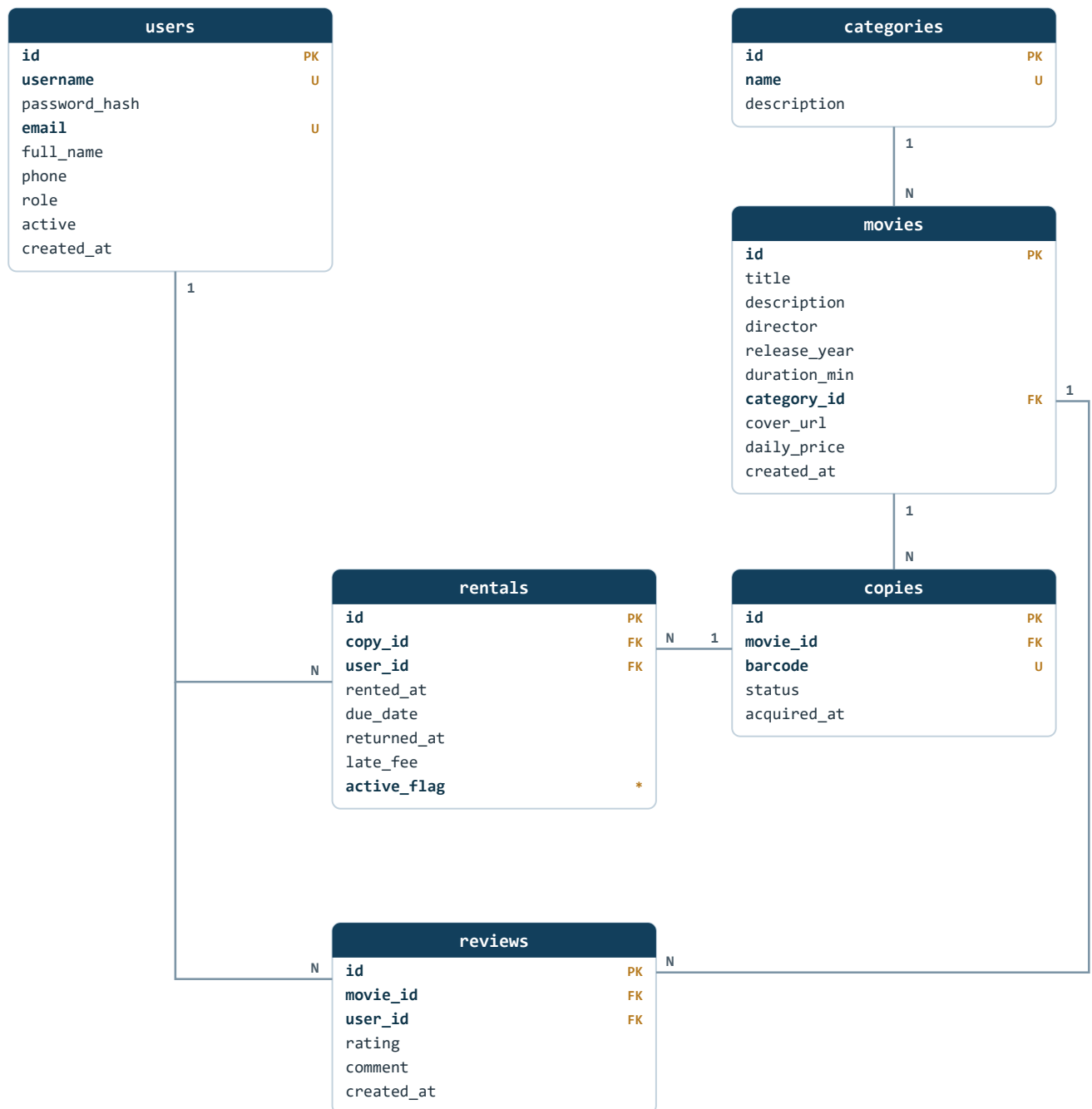
5.3 תרשים ERD



יחסי הטבלאות. * — **active_flag** היא עמודה מחושבת המשמשת לאילוץ **UNIQUE (copy_id, active_flag)**, המונע יותר מהשאלה פתוחה אחת לעותק.

5.4 העמודה **active_flag** — אכיפת השאלה יחידה

זהו הרכיב המרכזי בהבטחת נכונות המלאי, ולכן הוא מתואר בנפרד.



יחסי הטבלאות. * — `active_flag` היא עמודה מחושבת המשמשת לאילוף `UNIQUE (copy_id, active_flag)`, המונע יותר מהשאלה פתוחה אחת לעותק.

העמודה מקבלת את הערך `1` בהשאלה פתוחה, ו-`NULL` בהשאלה שהוחזרה. אילוץי ייחודיות ב-MYSQL מתעלמים מערכי `NULL`, ולכן:

- לעותק יכולות להיות אינספור השאלות שהוחזרו — כולן עם `NULL`.
- לעותק יכולה להיות לכל היותר השאלה פתוחה אחת — כי שני ערכי `1` לאותו `copy_id` יפרו את האילוץ.

התוצאה: השאלה כפולה של אותו עותק היא בלתי אפשרית ברמת בסיס הנתונים, ולא רק ברמת הקוד. גם באג עתידי בשכבת השירות, או ריצה של שני שרתי יישומים במקביל, לא יוכלו לעקוף אותה.

5.5 אינדקסים

אינדקס	טבלה	מטרה
<code>idx_copies_movie_status</code>	<code>copies</code>	<code>(movie_id, status)</code> — משרת את שאילתת איתור העותק הפנוי, השאילתה החמה במערכת.
<code>idx_rentals_user</code>	<code>rentals</code>	היסטוריית לקוח.
<code>idx_rentals_open</code>	<code>rentals</code>	איתור השאלות פתוחות ואיחורים.
<code>uq_rentals_copy_active</code>	<code>rentals</code>	אכיפת השאלה פתוחה יחידה (ראו 5.4).
<code>idx_movies_category</code>	<code>movies</code>	סינון לפי קטגוריה.

אין אינדקס **FULLTEXT**. ההסבר בפרק 15.

6. מבנה החבילות

`il.ac.openu.sartia`

<code>model</code>	9	אובייקטי התחום
<code>dao</code>	9	גישה לנתונים וניהול טרנזקציות
<code>service</code>	4	לוגיקה עסקית
<code>exception</code>	5	חריגות התחום
<code>web</code>	9	Backing Beans, סינון הרשאות, אתחול
<code>admin</code>	2	מסכי הניהול
<code>rest</code>	4	משאבי JAX-RS וממפי שגיאות
<code>dto</code>	4	אובייקטי תשובה ל-JSON
<code>util</code>	2	סיסמאות והגדרות

סה"כ 48 מחלקות. חלוקת החבילות משקפת את השכבות — קל לראות מהמבנה בלבד היכן נמצאת כל אחריות.

7. שכבת המודל

חבילת `il.ac.openu.sartia.model`. אובייקטים פשוטים (POJO) המייצגים את התחום. אינם תלויים בשום מסגרת עבודה.

User

חשבון במערכת. שדות: `id`, `username`, `passwordHash`, `email`, `fullName`, `phone`, `role`, `active`, `createdAt`.

מתודה	תיאור
<code>isAdmin()</code>	האם החשבון בעל הרשאות ניהול

תמצית הסיסמה נשמרת על האובייקט מפני ששכבת ה-DAO זקוקה לה לאימות, אך היא **נמחקת מיד לאחר אימות מוצלח** ואינה מגיעה לשכבת התצוגה.

Movie

<code>categoryId</code> , <code>durationMinutes</code> , <code>releaseYear</code> , <code>director</code> , <code>description</code> , <code>title</code> , <code>dailyPrice</code> , <code>coverUrl</code>	כותר בקטלוג. שדות מתמידים:
<code>reviewCount</code> , <code>averageRating</code> , <code>availableCopies</code> , <code>totalCopies</code> , <code>categoryName</code>	שדות נגזרים (מחושבים בשאילתה, אינם נשמרים):

מתודה	תיאור
<code>isAvailable()</code>	האם קיים עותק פנוי כרגע
<code>getFormattedRating()</code>	דירוג לתצוגה, או <code>null</code> אם טרם דורג

Copy

עותק פיזי. שדות: `acquiredAt` , `status` , `barcode` , `movieId`

Rental

השאלה. שדות: `copyId` , `userId` , `rentedAt` , `dueDate` , `returnedAt` , `lateFee`, ושדות מצורפים לתצוגה.

מתודה	תיאור
<code>isOpen()</code>	האם העותק עדיין אצל הלקוח
<code>isOverdue()</code>	פתוחה וגם עברה את תאריך ההחזרה
<code>getDaysOverdue()</code>	מספר ימי האיחור, או 0
<code>getProjectedLateFee()</code>	הקנס שייגבה בהחזרה עכשיו

`getProjectedLateFee()` מחזיר תמיד ערך בדיוק של שתי ספרות אחרי הנקודה. `BigDecimal.equals` משווה גם את הדיוק, ולכן 0 ו-0.00 אינם שווים בו — קיבוע הדיוק מונע השוואות שגויות. **הצורך בכך התגלה בבדיקת יחידה.**

CopyStatus, Role, Category, Review

אובייקטים ומונים פשוטים. `CopyStatus` נושא תווית תצוגה (`Lost` / `Rented` / `Available`) לצד שם הקבוע, כדי שהמסך לא יציג את שם קבוע ה-enum עצמו.

MovieSearchCriteria

מיכל לקריטריוני החיפוש שהמשתמש בחר, המועבר כאובייקט אחד מדף ה-JSF ועד ל-DAO.

מתודה	תיאור
<code>getOffset()</code>	היסט לפסוקית <code>LIMIT</code> . מגן מפני היסט שלילי
<code>hasKeyword()</code>	האם הוזן מונח חיפוש שאינו ריק
<code>resetPaging()</code>	חזרה לעמוד הראשון בעת שינוי סינון

העברת אובייקט קריטריונים במקום רשימת פרמטרים ארוכה שומרת על חתימת החיפוש יציבה ככל שנוספים מסננים, ומאפשרת ל-DAO לבנות את פסוקית ה-`WHERE` לפי השדות שמולאו בפועל.

8. שכבת הנתונים

חבילת `il.ac.openu.sartia.dao`.

Database

מחזיקה את מאגר החיבורים ומגדירה כיצד מריצים עבודה מולו.

מתודה	תיאור
<code>init()</code>	בונה את המאגר מקובץ ההגדרות
<code>shutdown()</code>	סוגר את המאגר בעת הורדת היישום
<code>inTransaction(work)</code>	מריץ יחידת עבודה בטרנזקציה אחת
<code>inTransaction(isolation, work)</code>	כנ"ל ברמת בידוד מפורשת
<code>runInTransaction(work)</code>	גרסה לעבודה שאינה מחזירה ערך
<code>readOnly(work)</code>	קריאה בלבד, ללא טרנזקציה מפורשת

גבולות הטרנזקציה מרוכזים כאן. כל יחידת עבודה מקבלת `Connection` ומובטח לה שהוא יבוצע (`commit`) או יבוטל (`rollback`) וייסגר בדיוק פעם אחת, בכל מסלול יציאה. מכאן ש-**ה-DAO מקבלים את החיבור כפרמטר ואינם פותחים חיבור משלהם** – זה מה שמאפשר לשירות לצרף כמה קריאות DAO לטרנזקציה אטומית אחת.

ניסיון חוזר בקיפאון (deadlock): נעילה ברמת שורה מאפשרת לשתי טרנזקציות הנוגעות באותן שורות בסדר שונה להיכנס לקיפאון; InnoDB פותר זאת בביטול אחת מהן. זהו מצב חולף וצפוי ולא באג, ולכן `inTransaction` מנסה שוב עד שלוש פעמים עם ההשגה עולה, ורק אז נכשל.

רמת הבידוד היא READ COMMITTED ולא ברירת המחדל REPEATABLE READ. הנימוק בפרק 12.

`runInTransaction` נושא שם שונה ולא עומס (`overload`) של `inTransaction` במכוון: ביטוי למדה שגופו קריאה למתודה חסרת ערך מחזר מתאים לשתי החתימות, ולכן עומס היה יוצר עמימות דווקא בנקודות הקריאה הזקוקות לו.

RowMapper

ממשק פונקציונלי הממיר שורת `ResultSet` לאובייקט תחום, עם מתודות עזר סטטיות `list`, `single`, ומתודות קריאה מודעות ל-NULL (`date`, `dateTime`, `nullableDouble`, `nullableInt`).

`ResultSet.getInt` מחזיר 0 גם עבור הערך 0 וגם עבור NULL; בעמודות כמו `release_year` ההבחנה משמעותית, ולכן מתודות אלה בודקות `wasNull()`.

MovieDao

תיאור	מתודה
סרט בודד עם נתוני מלאי ודירוג	<code>findById(conn, id)</code>
חיפוש בקטלוג עם עימוד	<code>search(conn, criteria)</code>
מספר התוצאות הכולל, לצורך העימוד	<code>countMatching(conn, criteria)</code>
תחזוקת הקטלוג	<code>insert / update / delete</code>

בניית שאילתת החיפוש: פסוקית ה-`WHERE` נבנית מהקריטריונים שמולאו בפועל. **רק מחרוזות SQL קבועות משורשרות** – כל ערך שמקורו במשתמש מועבר כפרמטר קשור (?), ועמודת המיון נבחרת ממונה (`enum`). לפיכך אין דרך שבה קלט משתמש ישנה את משמעות השאילתה.

נתוני המלאי (`available_copies`, `total_copies`) מחושבים בתת-שאילתות ולא נשמרים כעמודה. מונה שמור היה מחייב עדכון בכל טרנזקציית השאלה והחזרה, והיה שגוי ברגע שעדכון אחד היה מתפספס; ערך נגזר אינו יכול לסטות.

CopyDao

תיאור	מתודה
תופס עותק פנוי אחד לטרנזקציה הקוראת	<code>lockAvailableCopy(conn, movieId)</code>
שינוי סטטוס מותנה במצב הקודם	<code>updateStatus(conn, id, expected, new)</code>
תחזוקת מלאי	<code>insert</code> , <code>countAvailable</code> , <code>findByMovie</code>

`lockAvailableCopy` היא השאילתה החשובה במערכת ומתוארת בפרק 12.

`updateStatus` מקבלת את הסטטוס **הצפוי** והופכת את העדכון למותנה: השורה תיכתב רק אם היא עדיין במצב שהקורא סבר שהיא בו. מספר השורות שהשתנו מדווח לקורא, ובכך הוא יודע אם הפעולה שלו היא זו שביצעה את המעבר.

תיאור	מתודה
פתיחת השאלה	<code>insert(conn, copyId, userId, dueDate)</code>
סגירת השאלה. מותנית ב- <code>returned_at IS NULL</code>	<code>close(conn, rentalId, lateFee)</code>
היסטוריה והשאלות פעילות	<code>findOpenByUser</code> , <code>findByUser</code>
למסכי הניהול	<code>findOverdue</code> , <code>findAllOpen</code>
לבדיקת המכסה	<code>countOpenByUser</code>
האם הלקוח כבר מחזיק בכותר	<code>hasOpenRentalOfMovie</code>
האם רשאי לכתוב ביקורת	<code>hasEverRentedMovie</code>

`close` מותנית ב- `returned_at IS NULL` ולכן היא **אידמפוטנטית**: שליחה כפולה של הטופס, או שני מנהלנים המטפלים באותה החזרה, יעדכנו אפס שורות בפעם השנייה במקום לדרוס את חותמת ההחזרה המקורית.

UserDao

תיאור	מתודה
שליפה	<code>findAll</code> , <code>findById</code> , <code>findByUsername</code>
בדיקה מקדימה להודעה ידיונית	<code>emailExists</code> , <code>usernameExists</code>
נעילת שורת הלקוח – ראו פרק 12	<code>lockUser(conn, userId)</code>
תחזוקה	<code>update</code> , <code>insert</code>

ReviewDao

`save` משתמשת ב- `INSERT ... ON DUPLICATE KEY UPDATE` כדי שהוספה או עדכון ייפתרו בהצהרה אחת מול אילוץ הייחודיות. קריאה ואז הסתעפות ב-Java הייתה מותרת חלון שבו שתי שליחות מקבילות מחליטות שתיהן להוסיף.

DataAccessException

עוטפת `SQLException` כדי שהשכבות שמעל ה-DAO לא יידרשו לייבא טיפוס JDBC.

9. שכבת הלוגיקה העסקית

חבילת `il.ac.openu.sartia.service`. כל המחלקות `@ApplicationScoped`.

תיאור	מתודה
השאלת עותק. מתוארת במלואה בפרק 12	<code>rent(userId, movieId)</code>
החזרה, מחזירה את הקנס	<code>returnRental(rentalId, requestedBy, isAdmin)</code>
סימון עותק כאבוד	<code>markCopyLost(rentalId)</code>
שאלות לקוח	<code>openRentalsFor</code> , <code>historyFor</code>
שאלות ניהול	<code>overdueRentals</code> , <code>allOpenRentals</code>
האם להציג כפתור השאלה פעיל	<code>canRent(userId, movieId)</code>

`canRent` היא ייעוץ לתצוגה בלבד. היא קוראת ללא נעילה, ולכן תשובתה עלולה להתיישן עד שהלקוח ילחץ. `rent` בודקת הכול מחדש תחת נעילה והיא הסמכות.

`returnRental` מקבלת את מבקש הפעולה: לקוח רשאי להחזיר רק השאלה שלו, מנהלן — כל השאלה. ניסיון להחזיר השאלה של אחר מדווח כ"לא נמצא" ולא כ"אין הרשאה", כדי שהתשובה לא תאשר את קיומו של מזהה ההשאלה.

CatalogService

תיאור	מתודה
עמוד תוצאות + מספר תוצאות כולל	<code>search(criteria)</code>
שליפה	<code>copiesOf(id)</code> , <code>categories()</code> , <code>findMovie(id)</code>
יצירת סרט + עותקים בטרנזקציה אחת	<code>createMovie(movie, initialCopies)</code>
תחזוקה	<code>deleteMovie</code> , <code>addCopies</code> , <code>updateMovie</code>

מחלקת `SearchResult` המקוננת מחזירה עמוד תוצאות יחד עם מטא-נתוני העימוד. היא נכתבה כמחלקה עם Getters ולא כ-`record`, מפני ש-EL מאתר תכונה לפי `getX()` בעוד `record` חושף `x()`. הצורך בכך התגלה בזמן הרצה.

`createMovie` יוצרת את הסרט ואת העותקים בטרנזקציה אחת — לא ייתכן סרט בקטלוג ללא עותקים מאחוריו. `deleteMovie` מסרבת למחוק סרט שעותק שלו מושאל.

UserService

תיאור	מתודה
יצירת חשבון לקוח	<code>register(...)</code>
אימות פרטים	<code>authenticate(username, password)</code>
ניהול	<code>setActive</code> , <code>findAll</code> , <code>findById</code>

בדיקות הייחודיות המוקדמות נועדו להודעה ידידותית המצביעה על השדה הבעייתי. **הן אינן ההבטחה** – שתי הרשמות מקבילות יכולות לעבור אותן – ולכן גם הפרת אילוץ הייחודיות נתפסת ומדווחת באותה צורה.

`authenticate` מחשבת תמצית סיסמה גם כאשר שם המשתמש אינו קיים. חזרה מיידית הייתה הופכת חשבון שאינו קיים למהיר יותר לדחייה מסיסמה שגויה, והפרש הזמן הזה מספיק כדי לגלות אילו שמות משתמש רשומים.

ReviewService

`submit` מוודאת שהלקוח אכן השאיל את הסרט בעבר. ללא כלל זה הדירוגים המוצגים בקטלוג לא היו אומרים דבר על הסרטים.

חבילת החריגות

RuntimeException	
מחלקה מופשטת – בסיס לכל חריגות התחום	BusinessException
HTTP 400 – קלט לא תקין	ValidationException
HTTP 401 – אימות נכשל	AuthenticationException
HTTP 404 – משאב לא נמצא	NotFoundException
HTTP 409 – התנגשות מצב	ConflictException

הודעות החריגות כתובות באנגלית ומיועדות להצגה ישירה למשתמש, ללא תרגום בשכבת התצוגה. `BusinessException` נבדלת מ- `DataAccessException`: הראשונה היא תוצאה צפויה ("הסרט אזל"), השנייה היא תקלה במערכת.

10. שכבת התצוגה

חבילות `il.ac.openu.sartia.web` ו- `web.admin`.

דף	Bean	תיאור
catalog.xhtml	CatalogBean	קטלוג, חיפוש, סינון, עימוד
movie.xhtml	MovieDetailBean	פרטי סרט, השאלה, ביקורות
login.xhtml	LoginBean	כניסה
register.xhtml	RegisterBean	הרשמה
my-rentals.xhtml	MyRentalsBean	השאלות והיסטוריה
admin/movies.xhtml	MovieAdminBean	ניהול קטלוג ומלאי
admin/returns.xhtml	ReturnsBean	השאלות פתוחות והחזרות
error.xhtml	—	דף שגיאה אחיד
WEB-INF/templates/layout.xhtml	—	תבנית משותפת

תחומי החיים של ה-Beans

תחום	מחלקות	נימוק
@SessionScoped	SessionBean	זהות המשתמש נשמרת לאורך הביקור
@ViewScoped	CatalogBean, MovieDetailBean, MyRentalsBean, ReturnsBean, MovieAdminBean	המצב שורד מעבר בין עמודים ונעלם ביציאה מהמסך
@RequestScoped	LoginBean, RegisterBean	טופס חד-פעמי

הבחירה ב- `@ViewScoped` לקטלוג מכוונת: `@RequestScoped` היה מאפס את המסננים בכל לחיצה על "העמוד הבא", ו- `@SessionScoped` היה ממשיך להחיל אותם גם למחרת.

SessionBean

מחזיקה את זהות המשתמש. שם ה-Bean הוא `userSession` ולא `session`, מפני ש- `session` שמור ל-Producer מובנה של JSF וגורם לעמימות בפריסה.

`login()` מסובבת את מזהה ה-Session באמצעות `changeSessionId()` כהגנה מפני Session Fixation.

הערה על באג שהתגלה ותוקן: המימוש הראשון ביטל את ה-Session ויצר אחד חדש (`invalidate()` ואז `getSession(true)`). ביטול Session הורס גם את כל ה-Beans מתחום ה-Session — כולל `SessionBean` עצמה — ולכן ההשמה שבאה אחריו נכתבה למופע שהמכלול כבר השליכה, והמשתמש נראה כמי שלא התחבר כלל. `changeSessionId()` מנפיקה מזהה חדש תוך שמירת ה-Session ותכולתו, וזו הדרך הנכונה.

AuthFilter

רכיב Servlet Filter המגן על `/my-rentals.xhtml` ועל `/admin/*`.

האכיפה נמצאת ב-Filter ולא בכל דף מפני שבדיקה שצריך לזכור לכתוב בכל מסך חדש היא בדיקה שתישכח. כאן דף מוגן מעצם כתובתו. הקישורים בתפריט משקפים את החוק ואינם אופפים אותו.

משתמש שאינו מחובר מופנה למסך הכניסה עם היעד המקורי בפרמטר `returnTo`.

StartupListener

מאתחל את מאגר החיבורים בעליית היישום וסוגר אותו בהורדתו — כך המשאבים משתחררים ולא נדלפים בכל פריסה מחדש.

11. שכבת ה-REST

חבילות `il.ac.openu.sartia.rest` ו-`rest.dto`.

מחלקה	תפקיד
<code>RestApplication</code>	קובעת את שורש הממשק ל- <code>/api</code>
<code>MovieResource</code>	חיפוש, פרטי סרט, ביקורות
<code>CategoryResource</code>	רשימת קטגוריות
<code>ErrorMappers</code>	ממיר חריגות לקודי HTTP

אובייקטי ה-DTO

`MovieDto`, `CategoryDto`, `ReviewDto`, `PageDto` — כולם `record`.

הממשק מחזיר DTO ולא את אובייקט התחום, כדי שפורמט התשובה יהיה החלטה מפורשת ולא תוצר לוואי של סכמת הנתונים. שינוי שם עמודה אינו שובר לקוח, ושדות פנימיים אינם דולפים. `ReviewDto` נושא את שם המבקר אך לא את מזההו — ממשק ציבורי אינו אמור לשמש למיפוי ציבור הלקוחות.

הגבלת הממשק לקריאה בלבד

פעולת השאלה משנה מלאי ומחייבת זיהוי משתמש. חשיפתה ב-REST הייתה מחייבת מנגנון הזדהות שני לצד ה-Session של דפי ה-JSF, ושגיאה עדינה באחד מהם הייתה פוגעת באבטחה. פעולות קריאה אינן נושאות סיכון כזה.

12. טיפול במקביליות

זהו הפרק המרכזי במסמך. הדרישה "המערכת לא תשאל סרט שעדיין לא הוחזר" היא הטענה היחידה בפרויקט שאי אפשר להוכיח בקריאת קוד בלבד — מרוץ או שקורה או שלא.

12.1 שני מרוצים שונים

בתכנון זהו שני מרוצים נפרדים, הדורשים הגנות שונות:

המרוץ	התרחיש	ההגנה
לקוחות שונים, אותו כותר	20 לקוחות לוחצים "השאל" על הסרט האחרון	נעילת שורת העותק + אילוף ייחודיות
לקוח אחד, בקשות מקבילות	לחיצה כפולה, או שתי לשוניות פתוחות	נעילת שורת הלקוח

12.2 מהלך פעולת ההשאלה

`RentalService.rent()` רצה כטרנזקציה אחת:

#	הפעולה	הקריאה	מה היא מגנה מפניו
1	נעילת שורת הלקוח	<code> userDao.lockUser()</code>	מרוץ מסוג 2 – בקשות מקבילות של אותו לקוח
2	שליפת הסרט	<code> movieDao.findById()</code>	
3	בדיקת מכסת ההשאלות	<code> rentalDao.countOpenByUser()</code>	
4	בדיקה שהכותר אינו כבר אצלו	<code> rentalDao.hasOpenRentalOfMovie()</code>	
5	נעילת עותק פנוי	<code> copyDao.lockAvailableCopy()</code>	מרוץ מסוג 1 – לקוחות שונים על אותו עותק
6	שינוי סטטוס ל-RENTED	<code> copyDao.updateStatus()</code>	
7	יצירת רשומת ההשאלה	<code> rentalDao.insert()</code>	אילוף הייחודיות במסד הנתונים

כל השלבים מבוצעים יחד או כלל לא.

12.3 נעילת העותק

```
SELECT id FROM copies
WHERE movie_id = ? AND status = 'AVAILABLE'
ORDER BY id LIMIT 1
FOR UPDATE SKIP LOCKED
```

- **FOR UPDATE** – נעילה בלעדית על השורה, המוחזקת עד סיום הטרנזקציה. טרנזקציה אחרת אינה יכולה לבחור את אותה שורה, ולכן אינה יכולה להחליט שהעותק פנוי.
- **SKIP LOCKED** – לדלג על שורות שכבר נעולות במקום להמתין להן. בלעדיה, עשרה לקוחות המשאילים כותר מבוקש היו נעמדים בתור זה אחרי זה; עם הפסקית, כל אחד פשוט לוקח את העותק הפנוי הבא.

12.4 נעילת הלקוח ורמת הבידוד

שני החוקים הנספרים **לכל לקוח** – המכסה, והאיסור להשאיל כותר שכבר אצלו – היו במימוש הראשון "קרא ואז פעל": שתי בקשות שהגיעו יחד קראו שתיהן את המצב הישן והסיקו שתיהן שהן בגדר החוקים.

הפתרון הוא נעילת שורת הלקוח כ**הצהרה הראשונה בטרנזקציה**. שתי נקודות עדינות:

1. **סדר הנעילות** – תמיד לקוח ואז עותק. סדר אחיד בכל הטרנזקציות הוא מה שמונע קיפאון הדדי.

2. **רמת הבידוד** — ברירת המחדל של MySQL היא REPEATABLE READ, שבה תצולם עקבי נקבע בקריאה הראשונה של הטרנזקציה וכל קריאה לא-נועלת אחריה מוגשת ממנו. טרנזקציה שנועלת שורה כדי להסתדר מול אחרות הייתה עדיין קוראת נתונים **מלפני הנעילה** ופועלת לפיהם. לכן טרנזקציות המערכת רצות ב-**READ COMMITTED**, שבה כל הצהרה רואה את המצב המעודכן.

הערה על באג שהתגלה ותוקן: הבדיקה `doubleSubmitByOneCustomerRentsOnce` נכשלה והראתה שלקוח יחיד השיג שתי השאלות של אותו סרט. הוספת נעילת הלקוח לבדה **לא הספיקה** — הבדיקה עדיין נכשלה, מפני שהקריאה לשליפת הסרט קדמה לנעילה וקיבעה את התצולם לפנייה. רק שינוי הסדר יחד עם המעבר ל-**READ COMMITTED** פתר את הבעיה. זו דוגמה לכך שנעילה נכונה אינה מספיקה אם רמת הבידוד אינה מתאימה לה.

12.5 שלוש שכבות הגנה

השכבה	מה היא מונעת
נעילת שורת הלקוח	בקשות מקבילות של אותו לקוח
נעילת שורת העותק	לקוחות שונים התופסים את אותו עותק
אילוץ הייחודיות <code>uq_rentals_copy_active</code>	כל השאר — כולל באג עתידי בקוד

ההגנה השלישית היא הבטחה ברמת בסיס הנתונים ואינה תלויה בנכונות הקוד. השאלתה בשלב 6 נכתבה מותנית (`status = AVAILABLE`) גם היא, כדי שאם ההבטחה תיחלש אי-פעם, הכשל יהיה רועש ולא שקט.

13. אבטחה

האיום	ההגנה
הזרקת SQL	<code>PreparedStatement</code> בכל שאילתה; רק מחרוזות קבועות משורשרות; עמודת מיון ממונה
גניבת סיסמאות ממאגר גנוב	PBKDF2-HMAC-SHA256, 210,000 סבבים, מלח אקראי לכל משתמש
התקפת תזמון על שם משתמש	חישוב תמצית גם עבור משתמש שאינו קיים
מניית שמות משתמש	הודעת כניסה אחידה
Session Fixation	<code>changeSessionId()</code> בעת ההתחברות
גניבת עוגיית Session בסקריפט	<code>http-only</code> על עוגיית ה-Session
XSS	JSF מקודד פלט אוטומטית
גישה לא מורשית	<code>AuthFilter</code> לפי כתובת, ולא הסתרת קישורים
חשיפת מבנה פנימי בשגיאה	דף שגיאה כללי; הפרטים ליומן בלבד

על תמצית הסיסמאות: SHA-256 גולמי מתוכנן להיות מהיר — בדיוק ההפך מהנדרש כאן, שכן כרטיס מסך מנסה מיליארדי סיסמאות בשנייה מול טבלה גנובה. PBKDF2 עולה במכוון 210,000 סבבים לכל ניחוש. המלח האקראי מבטיח ששני משתמשים שבחרו אותה סיסמה יקבלו תמציות שונות, ולכן טבלת קשת מחושבת מראש חסרת ערך.

ההשוואה נעשית ב- `MessageDigest.isEqual`, הבודקת תמיד את כל הבתים. `equals` רגיל חוזר ברגע שנמצא הבדל, והפרש הזמן הזה מאפשר לשחזר תמצית בית אחר בית.

14. בדיקות

33 בדיקות, כולן עוברות.

מחלקה	סוג	בדיקות	מה נבדק
<code>PasswordsTest</code>	יחידה	6	תמצית, מלח, קלט פגום, יוניקוד
<code>RentalTest</code>	יחידה	6	איחור, קנסות, מקרי קצה
<code>MovieSearchCriteriaTest</code>	יחידה	5	חשבון עימוד, הגנה מהיסט שלילי
<code>RentalLifecycleIT</code>	אינטגרציה	11	מחזור השאלה-החזרה, הרשאות, אזילת מלאי, מחיקה ורצף ברקודים
<code>ConcurrentRentalIT</code>	אינטגרציה	5	מרוצים תחת עומס אמיתי

בדיקות המקביליות

`ConcurrentRentalIT` מפעילה חוט לכל לקוח, מחזיקה את כולם על מחסום (`CountDownLatch`) ומשחררת אותם יחד:

הבדיקה	מה היא מוודאת
20 לקוחות, 3 עותקים	בדיוק 3 השאלות; 17 קיבלו הודעת התנגשות נקייה
15 לקוחות, עותק אחד	בדיוק השאלה אחת
10 לקוחות, 10 עותקים	כולם מצליחים — תחרות אינה גורמת לכשלים שווא
לקוח אחד, 8 שליחות	השאלה אחת בלבד
לקוח אחד מול המכסה	בדיוק המכסה, לעולם לא יותר

הבדיקות הורצו **חמש פעמים ברצף** ועברו בכולן, כדי לוודא שהתוצאה אינה מקריות תזמון.

שתי הבדיקות האחרונות גילו באגים אמיתיים בקוד (ראו 12.4). זו הסיבה שהבדיקות משולבות בבנייה הרגילה ולא מסומנות לדילוג: חבילת בדיקות המדלגת בשקט על הטענה המרכזית הייתה מדווחת הצלחה מבלי לבדוק דבר.

בדיקת קצה-לקצה

מריצה 22 בדיקות מול פריסה חיה: היא מושכת כל דף, מחלצת ממנו את ה-View State ואת שמות

השדות שנוצרו, ושולחת את הטופס בחזרה — בדיוק כפי שדפדפן עושה.

אחת מבדיקותיה מחפשת במכון את המחרוזת `atrix` — קטע מאמצע מילה ובאותיות קטנות. היא מאמתת בבת אחת שהחיפוש אינו תלוי-רישיות ושהוא התאמת תת-מחרוזת ולא התאמת תחילית, כלומר שההחלטה שתוארה בפרק 15.3 אכן ממומשת.

15. החלטות תכנון והנימוק להן

15.1 סרט מול עותק

ניתן היה לשמור מונה `available_count` על טבלת הסרטים. לא נעשה כך: מונה מחייב עדכון בכל השאלה והחזרה, ואינו יודע איזה דיסק אצל מי. מודל העותקים נותן מעקב מדויק וזמינות נגזרת שאינה יכולה לסטות.

15.2 JPA ולא JDB

הבחירה מכוונת. המנגנון המרכזי הוא `SELECT ... FOR UPDATE SKIP LOCKED` – נעילה מפורשת שהיא לב פתרון המקבילות. ניסוחה ישירות ב-SQL בהיר ומדויק יותר מאשר דרך הפשטה שיש להסביר מה היא מייצרת מתחת. הבחירה גם הופכת את גבולות הטרנזקציה למפורשים.

15.3 העדר אינדקס FULLTEXT

זו החלטה שהתקבלה על סמך ניסוי ולא על סמך הנחה.

אינדקס FULLTEXT ב-MYSQL מתאים **מילים שלמות או תחיליות שלהן**, ולכן אינו יכול להתאים קטע מאמצע מילה. בקטלוג הנוכחי המשמעות ישירה: חיפוש `batman` לא היה מוצא את `The Dark Knight`, וחיפוש חלקי כמו `budap` לא היה מגיע ל-`The Grand Budapest Hotel`. התאמת תת-מחרוזת אינה סובלת ממגבלה זו:

```
-- לא מחזיר דבר עבור קטע מאמצע מילה --
SELECT ... WHERE MATCH(title,...) AGAINST ('+budap*' IN BOOLEAN MODE);

-- מחזיר את השורה:
SELECT ... WHERE title LIKE '%budap%';
```

לכן החיפוש מבוסס על התאמת תת-מחרוזת, והאינדקס **הוסר** במקום להישאר בסכמה מבלי לשמש. בקטלוג בסדר גודל זה עלות הסריקה אינה מדידה.

הרקע ההיסטורי להחלטה. הקטלוג נבנה תחילה בעברית, ושם המגבלה הייתה חמורה הרבה יותר: בעברית ה"א הידיעה ומילות היחס נכתבות מחוברות לתחילת המילה, ולכן `המטריקס` הוא `ה` + `מטריקס`. חיפוש תחילית עבור `מטריקס` לעולם לא היה מוצא את `המטריקס` – שהוא בדיוק מה שלקוח מקליד. הדבר נבדק ישירות מול בסיס הנתונים, וזו הבדיקה שהובילה להסרת האינדקס. המעבר לקטלוג באנגלית מחליש את הנימוק אך אינו מבטל אותו, שכן מגבלת התחילית נותרת בעינה. אילו הקטלוג היה גדל לעשרות אלפי כותרים, הפתרון הנכון היה מנוע חיפוש חיצוני – ולא אינדקס שאינו מוצא את מה שמחפשים.

15.4 חוקים עסקיים כהגדרות

תקופת ההשאלה והמכסה נמצאות ב-`sartia.properties` וניתנות לדריסה בפרמטר הרצה. הן מדיניות ולא לוגיקה: שינוי מדיניות אינו אמור לחייב בנייה מחדש.

15.5 עמידות בפני קלט לא תקין

הקלט נבדק בשלוש נקודות, בכוונה:

1. **בדפדפן** – סימון שדות חובה. נוחות בלבד, אינה הגנה.

2. בשכבת השירות — בדיקות תוכן. זו האכיפה.

3. בבסיס הנתונים — אילוצים, מפתחות זרים ו-`CHECK`. רשת הביטחון האחרונה.

15.6 תקלות שהתגלו בבדיקה ותוקנו

הפרויקט עבר סקירת קוד שיטתית לאחר השלמתו. להלן התקלות המהותיות שנמצאו ותוקנו, מפני שהן מדגימות היכן ההנחות היו שגויות:

התקלה	מה קרה בפועל	התיקון
השאלה כפולה ללקוח אחד	בדיקת המקביליות הראתה שלחיצה כפולה מייצרת שתי השאלות. הבדיקות "לקוח קרא ואז פעל" לא היו מוגנות.	נעילת שורת הלקוח + מעבר ל- <code>READ-COMMITTED</code> (פרק 12.4)
מחיקת סרט עם היסטוריה	הבדיקה מנעה מחיקה רק כאשר עותק מושאל. סרט שהושאל והוחזר עבר את הבדיקה ואז נכשל על מפתח זר — והמנהלן קיבל דף שגיאה במקום הסבר.	בדיקה נוספת לקיום רשומות השאלה כלשהן
ברקוד כפול לאחר פער ברצף	המתודה <code>maxBarcodeSequence</code> הבטיחה בשמה את המספר הגבוה ביותר, אך ביצעה <code>COUNT(*)</code> . חיסור עותק מהאמצע היה גורם להנפקת ברקוד קיים.	שאיילתה המחשבת <code>MAX</code> אמיתי
עדכון סטטוס עותק שנכשל בשקט	בהחזרה ובסימון כאבוד, תוצאת <code>updateStatus</code> לא נבדקה. כשל היה משאיר את העותק במצב שאינו תואם את ההשאלה.	בדיקת התוצאה וביטול הטרנזקציה
התחברות שלא נשמרה	<code>SessionBean.login</code> ביטלה את ה- <code>Session</code> — פעולה ההורסת גם את ה- <code>Bean</code> עצמו — ולכן ההשמה שאחריה אבדה.	<code>changeSessionId()</code> (פרק 10)

בנוסף הוסרו מספר מתודות שלא נקראו מאף מקום, ותוקנו הערות תיעוד שתיארו התנהגות שהקוד לא ביצע בפועל. לכל אחד משלושת התיקונים הראשונים נוספה בדיקת רגרסיה.

נספח א' – קבצים מרכזיים

קובץ	תוכן
db/schema.sql	סכמת בסיס הנתונים
db/seed.sql	נתוני הדגמה
src/main/resources/sartia.properties	הגדרות חיבור וחוקים עסקיים
src/main/webapp/images/covers/	תמונות העטיפה, נארזות לתוך ה-WAR
src/main/webapp/WEB-INF/web.xml	הגדרות היישום
src/main/webapp/WEB-INF/beans.xml	הגדרות CDI
src/main/webapp/WEB-INF/faces-config.xml	הגדרות JSF
scripts/smoke-test.sh	בדיקת קצה-לקצה
README.md	הוראות התקנה והרצה

נספח ב' – היקף הקוד

רכיב	שורות
קוד Java (ראשי)	4,660~
דפי XHTML	1,110~
קוד בדיקות	890~
CSS	385~
SQL	350~

48 מחלקות, 9 דפי Java, 6 XHTML טבלאות, 33 בדיקות, 28 תמונות עטיפה.